

CSCI-GA.1170-003 Fundamental Algorithms

Problem Set 3

Varun Sahni
N10338688
NetID: vs3606

September 30, 2026

Problem 3-1 (Recurrences and Matrix Exponentiation)

The sequence is defined by

$$F_0 = 1, \quad F_1 = 1, \quad F_2 = 2, \quad F_i = F_{i-1} + 5F_{i-2} + 3F_{i-3} \quad (i > 2).$$

(a) Start with a general 3×3 matrix

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}.$$

Multiplying it by the state vector gives

$$\begin{aligned} A \begin{pmatrix} F_i \\ F_{i-1} \\ F_{i-2} \end{pmatrix} &= \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} F_i \\ F_{i-1} \\ F_{i-2} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}F_i + a_{12}F_{i-1} + a_{13}F_{i-2} \\ a_{21}F_i + a_{22}F_{i-1} + a_{23}F_{i-2} \\ a_{31}F_i + a_{32}F_{i-1} + a_{33}F_{i-2} \end{pmatrix}. \end{aligned}$$

The required product is

$$\begin{pmatrix} F_{i+1} \\ F_i \\ F_{i-1} \end{pmatrix}.$$

The recurrence, with its index shifted by one, is

$$F_{i+1} = F_i + 5F_{i-1} + 3F_{i-2}.$$

Matching the coefficients in the first coordinate gives

$$a_{11} = 1, \quad a_{12} = 5, \quad a_{13} = 3.$$

Matching the second coordinate to F_i gives

$$a_{21} = 1, \quad a_{22} = 0, \quad a_{23} = 0.$$

Matching the third coordinate to F_{i-1} gives

$$a_{31} = 0, \quad a_{32} = 1, \quad a_{33} = 0.$$

Hence one solution is

$$A = \begin{pmatrix} 1 & 5 & 3 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

Substitution gives

$$\begin{aligned} A \begin{pmatrix} F_i \\ F_{i-1} \\ F_{i-2} \end{pmatrix} &= \begin{pmatrix} F_i + 5F_{i-1} + 3F_{i-2} \\ F_i \\ F_{i-1} \end{pmatrix} \\ &= \begin{pmatrix} F_{i+1} \\ F_i \\ F_{i-1} \end{pmatrix}. \end{aligned}$$

(b) For $n \geq 2$, repeated application of part (a) gives

$$\begin{pmatrix} F_n \\ F_{n-1} \\ F_{n-2} \end{pmatrix} = A^{n-2} \begin{pmatrix} F_2 \\ F_1 \\ F_0 \end{pmatrix} = A^{n-2} \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}.$$

The cases $n = 0, 1, 2$ are returned directly. For $n > 2$, compute A^{n-2} by repeated squaring:

```
MATRIX-POWER(A, k):
    R = I_3
    B = A
    while k > 0:
        if k is odd:
            R = R * B
        k = floor(k / 2)
        if k > 0:
            B = B * B
    return R

FAST-F(n):
    if n == 0: return 1
    if n == 1: return 1
    if n == 2: return 2
    v = MATRIX-POWER(A, n - 2) * (2, 1, 1)^T
    return v[1]
```

The loop invariant is

$$RB^k = A^{n-2},$$

where k denotes the current exponent. Initially, $RB^k = I_3 A^{n-2} = A^{n-2}$. If $k = 2q$ is even, replacing B by B^2 and k by q gives

$$R(B^2)^q = RB^{2q} = RB^k.$$

If $k = 2q + 1$ is odd, first replacing R by RB , and then replacing B by B^2 and k by q , gives

$$(RB)(B^2)^q = RB^{2q+1} = RB^k.$$

Thus the invariant is maintained. At termination $k = 0$, so

$$R = RB^0 = A^{n-2}.$$

The vector identity above then proves that FAST-F returns F_n .

After t loop iterations, the exponent is at most $\lfloor (n-2)/2^t \rfloor$. The loop stops when this is zero, which occurs after

$$t = \lfloor \log_2(n-2) \rfloor + 1$$

iterations. There is at most one squaring and one multiplication into R per iteration. Hence the total number of 3×3 matrix multiplications is at most

$$2t = 2\lfloor \log_2(n-2) \rfloor + 2 = O(\log n).$$

There is also at least one squaring per nonfinal iteration, so the count is $\Omega(\log n)$. Therefore it is

$$\boxed{\Theta(\log n)}.$$

(c) Suppose that

$$c_2 a^n \leq F_n \leq c_1 a^n$$

for one common constant $a > 1$.

Condition imposed by the upper-bound induction. Assume that $F_j \leq c_1 a^j$ for every $j < i$. Substitution into the recurrence gives

$$\begin{aligned} F_i &= F_{i-1} + 5F_{i-2} + 3F_{i-3} \\ &\leq c_1 a^{i-1} + 5c_1 a^{i-2} + 3c_1 a^{i-3} \\ &= c_1 a^{i-3} (a^2 + 5a + 3). \end{aligned}$$

The last expression is at most $c_1 a^i$ when

$$\begin{aligned} c_1 a^{i-3} (a^2 + 5a + 3) &\leq c_1 a^i, \\ a^2 + 5a + 3 &\leq a^3. \end{aligned}$$

Condition imposed by the lower-bound induction. Assume instead that $F_j \geq c_2 a^j$ for every $j < i$. Then

$$\begin{aligned} F_i &= F_{i-1} + 5F_{i-2} + 3F_{i-3} \\ &\geq c_2 a^{i-1} + 5c_2 a^{i-2} + 3c_2 a^{i-3} \\ &= c_2 a^{i-3} (a^2 + 5a + 3). \end{aligned}$$

The last expression is at least $c_2 a^i$ when

$$a^2 + 5a + 3 \geq a^3.$$

The upper and lower conditions hold simultaneously when

$$\begin{aligned} a^3 &= a^2 + 5a + 3, \\ a^3 - a^2 - 5a - 3 &= 0. \end{aligned}$$

The rational-root candidates are ± 1 and ± 3 . Testing $a = 3$ gives

$$3^3 - 3^2 - 5(3) - 3 = 27 - 9 - 15 - 3 = 0.$$

Factoring the polynomial gives

$$a^3 - a^2 - 5a - 3 = (a - 3)(a + 1)^2.$$

Its roots are 3, -1 , -1 , and the only root greater than 1 is

$$\boxed{a = 3}.$$

Thus

$$3^i = 3^{i-1} + 5 \cdot 3^{i-2} + 3 \cdot 3^{i-3}.$$

Deriving the best upper constant c_1 . The induction has three base cases because the recurrence uses the previous three terms. Each base case gives a necessary constraint:

$$\begin{aligned} n = 0 : \quad F_0 &\leq c_1 3^0 \iff 1 \leq c_1, \\ n = 1 : \quad F_1 &\leq c_1 3^1 \iff \frac{1}{3} \leq c_1, \\ n = 2 : \quad F_2 &\leq c_1 3^2 \iff \frac{2}{9} \leq c_1. \end{aligned}$$

All three must hold, so

$$c_1 \geq \max \left\{ 1, \frac{1}{3}, \frac{2}{9} \right\} = 1.$$

The smallest valid upper-bound constant is

$$\boxed{c_1 = 1}.$$

Deriving the best lower constant c_2 . The three lower-bound base cases similarly require

$$\begin{aligned} n = 0 : \quad F_0 &\geq c_2 3^0 \iff c_2 \leq 1, \\ n = 1 : \quad F_1 &\geq c_2 3^1 \iff c_2 \leq \frac{1}{3}, \\ n = 2 : \quad F_2 &\geq c_2 3^2 \iff c_2 \leq \frac{2}{9}. \end{aligned}$$

All three must hold, so

$$c_2 \leq \min \left\{ 1, \frac{1}{3}, \frac{2}{9} \right\} = \frac{2}{9}.$$

For a lower bound, the best constant is the largest permitted one. Hence

$$\boxed{c_2 = \frac{2}{9}}.$$

Inductive verification of the upper bound. The base cases are

$$F_0 = 1 \leq 1, \quad F_1 = 1 \leq 3, \quad F_2 = 2 \leq 9.$$

Assume $F_j \leq 3^j$ for every $j < i$, where $i \geq 3$. Then

$$\begin{aligned} F_i &= F_{i-1} + 5F_{i-2} + 3F_{i-3} \\ &\leq 3^{i-1} + 5 \cdot 3^{i-2} + 3 \cdot 3^{i-3} \\ &= 3^{i-3} (3^2 + 5 \cdot 3 + 3) \\ &= 3^{i-3} (9 + 15 + 3) \\ &= 3^{i-3} \cdot 27 \\ &= 3^i. \end{aligned}$$

Inductive verification of the lower bound. The base cases are

$$\begin{aligned} F_0 &= 1 \geq \frac{2}{9} \cdot 3^0 = \frac{2}{9}, \\ F_1 &= 1 \geq \frac{2}{9} \cdot 3^1 = \frac{2}{3}, \\ F_2 &= 2 \geq \frac{2}{9} \cdot 3^2. \end{aligned}$$

Assume $F_j \geq \frac{2}{9}3^j$ for every $j < i$. Then

$$\begin{aligned}
 F_i &= F_{i-1} + 5F_{i-2} + 3F_{i-3} \\
 &\geq \frac{2}{9}3^{i-1} + 5\left(\frac{2}{9}3^{i-2}\right) + 3\left(\frac{2}{9}3^{i-3}\right) \\
 &= \frac{2}{9}3^{i-3}(3^2 + 5 \cdot 3 + 3) \\
 &= \frac{2}{9}3^{i-3}(9 + 15 + 3) \\
 &= \frac{2}{9}3^{i-3} \cdot 27 \\
 &= \frac{2}{9}3^i.
 \end{aligned}$$

Therefore,

$$\boxed{\frac{2}{9}3^n \leq F_n \leq 3^n}, \quad \text{so} \quad \boxed{F_n = \Theta(3^n)}.$$

In particular, $\log_2 F_n = \Theta(n)$, so F_n requires $\Theta(n)$ bits to represent.

- (d) At a subproblem of index i , the matrix entries have $\Theta(i)$ bits. Under the bound stated in the problem, a 3×3 matrix multiplication takes

$$O(i^{\log_2 3})$$

time. Repeated squaring therefore gives

$$T(n) \leq T(\lceil n/2 \rceil) + cn^{\log_2 3}$$

for a constant $c > 0$. Let

$$U(n) = U(n/2) + \Theta(n^{\log_2 3}), \quad U(1) = \Theta(1),$$

be the corresponding upper-bound recurrence. For the Master Theorem,

$$a = 1, \quad b = 2, \quad f(n) = n^{\log_2 3}, \quad n^{\log_b a} = 1.$$

The polynomial condition holds with $\varepsilon = \log_2 3$. The regularity condition is

$$af(n/b) = f(n/2) = \frac{1}{3}f(n).$$

Case 3 gives

$$U(n) = \Theta(n^{\log_2 3}).$$

The final multiplication by $(2, 1, 1)^\top$ uses $O(n)$ -bit additions and does not change the bound. Consequently,

$$\boxed{T(n) = O(n^{\log_2 3})}.$$

If each Karatsuba multiplication is charged its tight $\Theta(i^{\log_2 3})$ running time, the same recurrence yields the tight bound $\Theta(n^{\log_2 3})$.

- (e) The naive algorithm computes $F_3, F_4, \dots, F_n$ in sequence. At step i , all operands have $\Theta(i)$ bits by part (c). Multiplication by the constants 3 and 5 takes only a constant number of additions, so the step costs $\Theta(i)$. Consequently,

```
NAIVE-F(n):  
    if n == 0: return 1  
    if n == 1: return 1  
    if n == 2: return 2  
    f0 = 1  
    f1 = 1  
    f2 = 2  
    for i = 3 to n:  
        f3 = f2 + 5*f1 + 3*f0  
        f0 = f1  
        f1 = f2  
        f2 = f3  
    return f2
```

After the iteration with index i , the variables $(f0, f1, f2)$ equal (F_{i-2}, F_{i-1}, F_i) . This is true before the first iteration, where they equal (F_0, F_1, F_2) , and the recurrence shows that one update preserves the claim. At termination, $f2 = F_n$, proving correctness.

For the running time, the summation is

$$\begin{aligned} T_{\text{naive}}(n) &= \Theta\left(\sum_{i=3}^n i\right) \\ &= \Theta\left(\frac{n(n+1)}{2} - 3\right) \\ &= \boxed{\Theta(n^2)}. \end{aligned}$$

This grows faster than the $O(n^{\log_2 3}) \approx O(n^{1.585})$ bound from part (d).

Problem 3-2 (Counting in a Grid)

For a submatrix, rows and columns are always understood to retain their original sorted order.

(a) Let $M[i][j]$ be an arbitrary entry. Since row 1 is sorted from left to right,

$$M[1][1] \leq M[1][j].$$

Since column j is sorted from top to bottom,

$$M[1][j] \leq M[i][j].$$

Combining the inequalities gives

$$M[1][1] \leq M[1][j] \leq M[i][j].$$

Thus $M[1][1]$ is no larger than any entry in the matrix, so the smallest element is

$$\boxed{M[1][1]}.$$

Since row i is sorted from left to right,

$$M[i][j] \leq M[i][n].$$

Since column n is sorted from top to bottom,

$$M[i][n] \leq M[m][n].$$

Combining the inequalities gives

$$M[i][j] \leq M[i][n] \leq M[m][n].$$

Thus $M[m][n]$ is no smaller than any entry in the matrix, so the largest element is

$$\boxed{M[m][n]}.$$

For valid $i < m$ and $j < n$,

$$M[i][j] \leq M[i][j+1] \leq M[i+1][j+1],$$

so

$$\boxed{M[i][j] \leq M[i+1][j+1]}.$$

Equality is possible because entries may repeat.

(b) The completed brute-force algorithm is

```

GRID-COUNT-A(M, m, n, key):
    count = 0
    for i = 1 to m:
        for j = 1 to n:
            if M[i][j] <= key:
                count = count + 1
    return count

```

After the first k entries in row-major order have been examined, $count$ equals the number of those k entries that are at most key . Initially $k = 0$ and $count = 0$. Each iteration examines the next entry and increments $count$ exactly when that entry qualifies, so the claim is maintained. When the loops terminate, $k = mn$, so $count$ is the desired answer.

The inner loop executes n times for each of m outer-loop iterations. Thus the comparison is executed

$$\underbrace{n + n + \cdots + n}_{m \text{ times}} = mn$$

times. The worst-case running time is therefore

$$\boxed{\Theta(mn)}.$$

(c) Let the four quadrants be as in the question.

If $M[i][j] \leq key$, then every element of Quadrant 2 is at most $M[i][j]$, and hence is at most key . Thus Quadrant 2 is known completely: it contributes exactly ij entries. Quadrants 1, 3, and 4 may still contain a mixture of qualifying and nonqualifying entries and must be explored.

If $M[i][j] > key$, then every element in Quadrant 4 is at least $M[i+1][j+1] \geq M[i][j] > key$. Thus Quadrant 4 contributes zero and need not be explored. Quadrants 1, 2, and 3 may still need to be explored.

Therefore,

test	known quadrant	quadrants still explored
$M[i][j] \leq key$	Q_2 contributes ij	Q_1, Q_3, Q_4
$M[i][j] > key$	Q_4 contributes 0	Q_1, Q_2, Q_3

- (d) Choose $i = m/2$ and $j = n/2$ whenever $m, n > 1$. Since m and n are powers of 2, each quadrant has

$$\frac{m}{2} \cdot \frac{n}{2} = \frac{N}{4}$$

entries. The recursive procedure is the following, where the parameters specify the current rectangular submatrix.

```
GRID-COUNT-DC(M, rows, columns, key):
  if the submatrix has one row:
    return the number of entries <= key by binary search
  if the submatrix has one column:
    return the number of entries <= key by binary search
  split rows and columns in half to form Q1, Q2, Q3, Q4
  x = bottom-right entry of Q2
  if x <= key:
    return |Q2| + GRID-COUNT-DC(Q1)
      + GRID-COUNT-DC(Q3)
      + GRID-COUNT-DC(Q4)
  else:
    return GRID-COUNT-DC(Q1)
      + GRID-COUNT-DC(Q2)
      + GRID-COUNT-DC(Q3)
```

By part (c), the omitted quadrant's contribution is known exactly.

Correctness. Induct on the number of entries. If the submatrix has one row or one column, its entries are sorted and binary search returns the length of the qualifying prefix. If both dimensions exceed 1, part (c) gives the exact contribution of the omitted quadrant. The three recursive quadrants are disjoint and smaller, so the induction hypothesis gives their exact counts. Their sum plus the known contribution is the count for the entire submatrix.

For a balanced grid, each call makes three recursive calls on subproblems of size $N/4$ and performs constant nonrecursive work:

$$T(N) = \begin{cases} \Theta(1), & N = 1, \\ 3T(N/4) + \Theta(1), & N > 1. \end{cases}$$

Here $a = 3$, $b = 4$, and $f(N) = \Theta(1)$. Since $f(N) = O(N^{\log_4 3 - \varepsilon})$ for a suitable $\varepsilon > 0$, this is Case 1 of the Master Theorem. Therefore

$$T(N) = \Theta(N^{\log_4 3}).$$

For an arbitrary rectangle, assume $m \leq n$. The recursion reaches a one-row base case after $\log_2 m$ levels. There are $3^{\log_2 m} = m^{\log_2 3}$ leaves, each containing n/m entries, so the total time is

$$O\left(m^{\log_2 3} \left(1 + \log_2 \frac{n}{m}\right)\right) = O\left((mn)^{\log_4 3}\right).$$

The bound uses $1 + \log_2 x = O(x^{\log_4 3})$ for $x \geq 1$. Square grids attain $\Theta(N^{\log_4 3})$, so this is the worst-case bound as a function of $N = mn$.

With $k = \log_4 N$, repeated substitution gives

$$\begin{aligned} T(N) &= 3T(N/4) + c \\ &= 3(3T(N/4^2) + c) + c \\ &= 3^2T(N/4^2) + c(1 + 3) \\ &\vdots \\ &= 3^k T(N/4^k) + c \sum_{r=0}^{k-1} 3^r \\ &= 3^{\log_4 N} T(1) + c \frac{3^k - 1}{3 - 1} \\ &= N^{\log_4 3} \Theta(1) + \Theta(3^k) \\ &= \Theta(N^{\log_4 3}). \end{aligned}$$

- (e) Maintain a current rectangle consisting of rows $r, \dots, m$ and columns $1, \dots, c$; its top-right entry is $M[r][c]$.

```

COUNT-LE(M, m, n, key):
    r = 1
    c = n
    count = 0
    while r <= m and c >= 1:
        if M[r][c] <= key:
            count = count + c
            r = r + 1
        else:
            c = c - 1
    return count

```

Loop invariant. At the start of every iteration:

- i. *count* is exactly the number of qualifying entries in rows $1, \dots, r - 1$ and columns $1, \dots, n$;
- ii. every entry in rows $r, \dots, m$ and columns $c + 1, \dots, n$ is known to be greater than *key*;
- iii. the only entries whose status has not yet been accounted for form the rectangle with rows $r, \dots, m$ and columns $1, \dots, c$.

Initialization. Initially $r = 1$, $c = n$, and *count* = 0. There are no processed rows and no removed columns, so all three statements hold.

Maintenance. Suppose first that $M[r][c] \leq \text{key}$. For each $1 \leq j \leq c$, row sortedness gives

$$M[r][j] \leq M[r][c] \leq \text{key}.$$

Thus exactly c qualifying entries are added. Entries in columns larger than c were already proved too large by the invariant, so row r is now completely accounted for; increasing r preserves all three statements.

Now suppose $M[r][c] > \text{key}$. For every $r \leq i \leq m$, column sortedness gives

$$M[i][c] \geq M[r][c] > \text{key}.$$

Thus no entry in the remaining part of column c qualifies. Decreasing c removes precisely this known-failing column and again preserves the invariant.

Termination. The loop stops when $r = m + 1$ or $c = 0$. In the first case every row has been counted. In the second case no unaccounted columns remain, and all removed columns contain only known-failing entries in the unprocessed rows. Therefore *count* is exactly the number of entries at most *key*.

For the running time, define the potential

$$\Phi = (m - r + 1) + c,$$

the number of remaining row and column boundaries. Initially $\Phi = m + n$. Every iteration increments r or decrements c , so it reduces Φ by exactly 1. Thus there are at most $m + n - 1$ iterations, each doing constant work. In recursive notation, the worst-case recurrence is

$$T(m, n) \leq \max\{T(m - 1, n), T(m, n - 1)\} + \Theta(1),$$

with $T(0, n) = T(m, 0) = \Theta(1)$. Each call decreases $m + n$ by one, giving

$$\boxed{T(m, n) = \Theta(m + n)}.$$

The bound is tight in the worst case because a monotone matrix and a suitable *key* can make the staircase alternate between row removals and column removals until it reaches the opposite boundary.

- (f) Count entries equal to key by subtracting the number strictly below key from the number at most key :

$$\#\{M[i][j] = key\} = \text{COUNTLE}(key) - \text{COUNTLT}(key).$$

```

COUNT-LE(M, m, n, key):
    r = 1
    c = n
    count = 0
    while r <= m and c >= 1:
        if M[r][c] <= key:
            count = count + c
            r = r + 1
        else:
            c = c - 1
    return count

COUNT-LT(M, m, n, key):
    r = 1
    c = n
    count = 0
    while r <= m and c >= 1:
        if M[r][c] < key:
            count = count + c
            r = r + 1
        else:
            c = c - 1
    return count

COUNT-EQUAL(M, m, n, key):
    atMost = COUNT-LE(M, m, n, key)
    below = COUNT-LT(M, m, n, key)
    return atMost - below

```

Every entry falls into exactly one of the three disjoint classes

$$x < key, \quad x = key, \quad x > key.$$

Consequently,

$$\begin{aligned} \#\{x \leq key\} &= \#\{x < key\} + \#\{x = key\}, \\ \#\{x = key\} &= \#\{x \leq key\} - \#\{x < key\}. \end{aligned}$$

Each call starts at the top-right corner. If its comparison succeeds, all c entries in the current row prefix satisfy the comparison, so it adds c and moves down. If the comparison fails, every remaining entry in column c also fails, so it moves left. Each iteration removes one row or one column, and each call takes $O(m + n)$ time. Hence COUNT-EQUAL takes $O(m + n)$ time.

For $key = 8$, the matrix is

$$M = \begin{pmatrix} 1 & 3 & 4 & 9 \\ 2 & 5 & 7 & 10 \\ 6 & 8 & 8 & 12 \\ 11 & 13 & 14 & 15 \end{pmatrix}.$$

Walkthrough of Count-LE($M, 4, 4, 8$).

step	r	c	$M[r][c]$	action	$count$
1	1	4	9	$9 > 8, c \leftarrow 3$	0
2	1	3	4	$4 \leq 8, count \leftarrow count + 3, r \leftarrow 2$	3
3	2	3	7	$7 \leq 8, count \leftarrow count + 3, r \leftarrow 3$	6
4	3	3	8	$8 \leq 8, count \leftarrow count + 3, r \leftarrow 4$	9
5	4	3	14	$14 > 8, c \leftarrow 2$	9
6	4	2	13	$13 > 8, c \leftarrow 1$	9
7	4	1	11	$11 > 8, c \leftarrow 0$	9

At step 2, the entries 1, 3, 4 are counted. At step 3, the entries 2, 5, 7 are counted. At step 4, the entries 6, 8, 8 are counted. Thus

$$\text{COUNTLE}(8) = 3 + 3 + 3 = 9.$$

Walkthrough of Count-LT($M, 4, 4, 8$).

step	r	c	$M[r][c]$	action	$count$
1	1	4	9	$9 \not\leq 8, c \leftarrow 3$	0
2	1	3	4	$4 < 8, count \leftarrow count + 3, r \leftarrow 2$	3
3	2	3	7	$7 < 8, count \leftarrow count + 3, r \leftarrow 3$	6
4	3	3	8	$8 \not\leq 8, c \leftarrow 2$	6
5	3	2	8	$8 \not\leq 8, c \leftarrow 1$	6
6	3	1	6	$6 < 8, count \leftarrow count + 1, r \leftarrow 4$	7
7	4	1	11	$11 \not\leq 8, c \leftarrow 0$	7

At steps 2 and 3, the entries 1, 3, 4 and 2, 5, 7 are counted. At step 6, only the entry 6 is counted. Thus

$$\text{COUNTLT}(8) = 3 + 3 + 1 = 7.$$

Therefore,

$$\#\{M[i][j] = 8\} = \text{COUNTLE}(8) - \text{COUNTLT}(8) = 9 - 7 = \boxed{2}.$$

- (g) For each row i , binary-search for the rightmost column j_i such that $M[i][j_i] \leq key$ (taking $j_i = 0$ if no such column exists). Then row i contributes j_i , so return $\sum_{i=1}^m j_i$. There are m binary searches, each taking $O(\log n)$ time, for a total of

```

RIGHTMOST-LE(row, n, key):
    low = 1
    high = n
    answer = 0
    while low <= high:
        mid = floor((low + high) / 2)
        if row[mid] <= key:
            answer = mid
            low = mid + 1
        else:
            high = mid - 1
    return answer

COUNT-LE-BINARY(M, m, n, key):
    count = 0
    for i = 1 to m:
        j = RIGHTMOST-LE(M[i], n, key)
        count = count + j
    return count

```

RIGHTMOST-LE initializes $answer = 0$, which represents that no qualifying entry has yet been found. If $row[mid] \leq key$, then every entry at or to the left of mid also satisfies the inequality. The algorithm sets $answer = mid$ and searches columns $mid + 1, \dots, high$ for a larger qualifying index. If $row[mid] > key$, then every entry at or to the right of mid also fails, so the algorithm continues in columns $low, \dots, mid - 1$. At termination, $answer$ is the rightmost qualifying column, or 0 if the row has no qualifying entry.

$$\Theta(m \log n).$$

One binary search obeys

$$B(n) = B(\lfloor n/2 \rfloor) + \Theta(1).$$

After k expansions its remaining interval has size at most $n/2^k$. Setting $n/2^k = 1$ gives $k = \log_2 n$, so $B(n) = \Theta(\log n)$. Repeating this for m rows gives $m\Theta(\log n) = \Theta(m \log n)$.

Correctness follows because each row is nondecreasing. Precisely the prefix

$$M[i][1], \dots, M[i][j_i]$$

is at most key . These row prefixes are disjoint and cover all qualifying entries, so their total size is $\sum_{i=1}^m j_i$.

The staircase method takes $O(m + n)$. For a roughly square matrix it is asymptotically better: $O(n)$ instead of $O(n \log n)$. Neither bound is uniformly smaller for every rectangle; if the matrix is very wide (for example, $m = 1$), row-wise binary search can be better. In general one may use the better of the two bounds, $O(\min\{m + n, m \log n\})$.

Problem 3-3 (Counting Profitable Trades)

A profitable pair is a pair (i, j) with $i < j$ and $A[j] > A[i]$.

- (a) Examine every pair of indices $i < j$, incrementing a counter exactly when $A[j] > A[i]$. There are $\binom{n}{2}$ such pairs and each comparison takes constant time, so the running time is

```
BRUTE-PROFITABLE-PAIRS(A, n):
    count = 0
    for i = 1 to n - 1:
        for j = i + 1 to n:
            if A[j] > A[i]:
                count = count + 1
    return count
```

Every pair (i, j) with $i < j$ occurs in exactly one loop iteration, and that iteration increments the counter exactly when the pair is profitable. Thus the returned count is correct. The exact number of iterations is

$$\begin{aligned} \sum_{i=1}^{n-1} (n-i) &= (n-1) + (n-2) + \cdots + 1 \\ &= \frac{n(n-1)}{2} \\ &= \Theta(n^2). \end{aligned}$$

Therefore the running time is

$$\boxed{\Theta(n^2)}.$$

- (b) Since every index in L precedes every index in R , a cross pair is profitable exactly when its values (l, r) satisfy $l < r$. Sort the values of L . For each $r \in R$, binary-search in sorted L for the first position whose value is at least r . The number of earlier positions is exactly the number of $l \in L$ with $l < r$, hence exactly the number of profitable cross pairs ending at this occurrence of r . Summing these counts over all occurrences in R gives the total number of profitable cross pairs.

```
CROSS-COUNT-BINARY(L, R):
    S = a sorted copy of L
    cross = 0
    for each occurrence r in R:
        k = first index in S with S[k] >= r
        if no such index exists:
            cross = cross + |S|
        else:
            cross = cross + (k - 1)
    return cross

PROFITABLE-PAIRS-BINARY(A):
    if |A| <= 1: return 0
    split A by position into L and R
    return PROFITABLE-PAIRS-BINARY(L)
        + PROFITABLE-PAIRS-BINARY(R)
        + CROSS-COUNT-BINARY(L, R)
```

Sorting L changes no cross-pair count: every element of L still comes from an index earlier than every element of R , and the test depends only on the two values. Searching for the first value *at least* r , rather than the first value greater than r , excludes equal prices.

The combine step costs $\Theta(n \log_2 n)$: sorting L costs $\Theta(n \log_2 n)$, and the $n/2$ binary searches cost $\Theta(n \log_2 n)$ in total. Thus

$$T(n) = 2T(n/2) + \Theta(n \log_2 n), \quad T(1) = \Theta(1).$$

For the Master Theorem,

$$a = 2, \quad b = 2, \quad n^{\log_b a} = n, \quad f(n) = \Theta(n \log_2 n).$$

For every $\varepsilon > 0$,

$$\frac{n \log_2 n}{n^{1-\varepsilon}} = n^\varepsilon \log_2 n \rightarrow \infty, \quad \frac{n \log_2 n}{n^{1+\varepsilon}} = \frac{\log_2 n}{n^\varepsilon} \rightarrow 0.$$

Thus $f(n)$ is neither $\Theta(n)$ nor polynomially smaller or larger than n . None of the three Master Theorem cases applies.

Let $n = 2^h$. At recursion level j , there are 2^j subproblems of size $n/2^j$. Their combined nonrecursive cost is

$$\begin{aligned} 2^j \Theta\left(\frac{n}{2^j} \log_2 \frac{n}{2^j}\right) &= \Theta\left(n \log_2 \frac{n}{2^j}\right) \\ &= \Theta(n(\log_2 n - j)). \end{aligned}$$

Summing the $h = \log_2 n$ internal levels gives

$$\begin{aligned} \sum_{j=0}^{h-1} \Theta(n(h-j)) &= \Theta\left(n \sum_{q=1}^h q\right) \\ &= \Theta\left(n \frac{h(h+1)}{2}\right) \\ &= \Theta(n(\log_2 n)^2). \end{aligned}$$

The n leaves add $\Theta(n)$, so

$$\boxed{T(n) = \Theta(n(\log_2 n)^2)}.$$

- (c) The following merge-sort variant returns both the number of profitable pairs and an ascending sorted copy.

```

COUNT-AND-SORT(A):
    if |A| <= 1:
        return (0, A)
    split A by position into L and R
    (pL, SL) = COUNT-AND-SORT(L)
    (pR, SR) = COUNT-AND-SORT(R)
    i = 1
    j = 1
    cross = 0
    S = empty list
    while i <= |SL| and j <= |SR|:
        if SL[i] < SR[j]:
            append SL[i] to S
            i = i + 1
        else:
            append SR[j] to S
            cross = cross + (i - 1)
            j = j + 1
    append all remaining entries of SL to S
    while j <= |SR|:
        append SR[j] to S
        cross = cross + |SL|
        j = j + 1
    return (pL + pR + cross, S)

```

When an element $S_R[j]$ is appended, exactly $i - 1$ elements from S_L have already been appended, and each is strictly smaller than $S_R[j]$. These are exactly the profitable cross pairs ending at that particular occurrence. If values are equal, the right value is appended first, so the equal left values are not counted.

- (d) The two recursive calls are on arrays of size $n/2$, and the merge and cross-pair count take $\Theta(n)$ time. Thus

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Here $a = 2$, $b = 2$, and $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$. This is Case 2 of the Master Theorem, so

$$T(n) = \Theta(n \log n).$$

Equivalently, level j has 2^j subproblems, each with merge cost $\Theta(n/2^j)$. Therefore every level costs

$$2^j \Theta(n/2^j) = \Theta(n).$$

There are $\log_2 n$ internal levels, plus n constant-time leaves, so

$$T(n) = \Theta(n) \log_2 n + \Theta(n) = \Theta(n \log n).$$

This improves on both the $\Theta(n^2)$ algorithm in part (a) and the $\Theta(n \log^2 n)$ algorithm in part (b).

- (e) For every input array A , COUNT-AND-SORT returns a pair $(P(A), S)$, where $P(A)$ is the number of profitable pairs in A and S is an ascending sorted permutation of A . The proof is by induction on $n = |A|$.

Base case. If $n \leq 1$, there are no pairs of distinct indices. The algorithm returns $(0, A)$; the count is correct and A is already sorted.

Induction hypothesis. For every array of length less than n , the algorithm returns its exact profitable-pair count and an ascending sorted permutation.

Recursive counts. Split A by position into L and R . By the induction hypothesis, the recursive calls return

$$(p_L, S_L) = (P(L), \text{sort}(L)), \quad (p_R, S_R) = (P(R), \text{sort}(R)).$$

Thus p_L counts exactly the profitable pairs whose two indices are in L , and p_R counts exactly those whose two indices are in R .

Sorted output. While both sorted lists are nonempty, the merge appends the smaller front value. If the two front values are equal, it appends the right value; either choice preserves nondecreasing order. Appending the unconsumed suffix after one list is exhausted therefore produces an ascending sorted permutation S of A .

Cross-pair count. Consider the moment at which the occurrence $r = S_R[j]$ is appended. The algorithm takes a left value only under the strict test

$$S_L[i] < S_R[j].$$

Consequently, the $i - 1$ left values already appended are strictly smaller than r .

If both lists are nonempty and the algorithm selects r , then

$$r \leq S_L[i].$$

Since S_L is sorted, every unconsumed left value is at least r . Hence exactly the first $i - 1$ elements of S_L form profitable cross pairs with this occurrence of r , and the update

$$\text{cross} \leftarrow \text{cross} + (i - 1)$$

counts all and only those pairs.

If S_L is exhausted, all $|S_L|$ left values are strictly smaller than each remaining right value, so each remaining right value contributes $|S_L|$. If S_R is exhausted, no right endpoint remains and the unconsumed left values contribute no new cross pairs.

When $S_L[i] = S_R[j]$, the right value is appended first and the equal left value is not among the first $i - 1$ elements. Equal prices are therefore not counted, as required by the strict condition $A[j] > A[i]$.

Every profitable pair belongs to exactly one of the disjoint classes

$$\text{inside } L, \quad \text{inside } R, \quad \text{crossing from } L \text{ to } R.$$

The returned count is therefore

$$p_L + p_R + \text{cross} = P(A).$$

Together with the sorted-output argument, this proves the induction claim.

- (f) The algorithm returns both the total profit of all profitable pairs and an ascending sorted copy of the input.

```

TOTAL-PROFIT-AND-SORT(A):
    if |A| <= 1:
        return (0, A)

    split A by position into L and R
    (profitL, SL) = TOTAL-PROFIT-AND-SORT(L)
    (profitR, SR) = TOTAL-PROFIT-AND-SORT(R)

    i = 1
    j = 1
    placedCount = 0
    placedSum = 0
    crossProfit = 0
    S = empty list

    while i <= |SL| and j <= |SR|:
        if SL[i] < SR[j]:
            append SL[i] to S
            placedCount = placedCount + 1
            placedSum = placedSum + SL[i]
            i = i + 1
        else:
            r = SR[j]
            append r to S
            crossProfit = crossProfit
                + placedCount*r - placedSum
            j = j + 1

    while i <= |SL|:
        append SL[i] to S
        i = i + 1

    while j <= |SR|:
        r = SR[j]
        append r to S
        crossProfit = crossProfit
            + placedCount*r - placedSum
        j = j + 1

    totalProfit = profitL + profitR + crossProfit
    return (totalProfit, S)

```

Cross-profit calculation. When a left value l is appended, $placedCount$ and $placedSum$ are updated so that

$$placedCount = c, \quad placedSum = s = \sum_{\substack{l \in S_L \\ l \text{ already placed}}} l.$$

When a right value r is appended, the already placed left values are exactly the left values

strictly smaller than r . Their total contribution with this occurrence of r is

$$\begin{aligned}\sum_{\substack{l \in S_L \\ l < r}} (r - l) &= \sum_{\substack{l \in S_L \\ l < r}} r - \sum_{\substack{l \in S_L \\ l < r}} l \\ &= cr - s \\ &= placedCount \cdot r - placedSum.\end{aligned}$$

The right value is selected first on a tie, so equal left values have not been placed and contribute nothing.

Correctness. For every array A , the algorithm returns $(Q(A), S)$, where

$$Q(A) = \sum_{\substack{i < j \\ A[j] > A[i]}} (A[j] - A[i])$$

and S is an ascending sorted permutation of A . The proof is by induction on $n = |A|$.

Base case. If $n \leq 1$, there are no pairs, so $Q(A) = 0$, and A is already sorted. The returned pair $(0, A)$ is correct.

Induction hypothesis. The claim holds for every array shorter than n .

Induction step. Split A by position into L and R . The induction hypothesis gives

$$\begin{aligned} (\text{profit}L, S_L) &= (Q(L), \text{sort}(L)), \\ (\text{profit}R, S_R) &= (Q(R), \text{sort}(R)). \end{aligned}$$

The merge appends the smaller front value, taking the right value first on a tie, and appends the remaining suffix after one list is exhausted. Hence S is an ascending sorted permutation of A .

Consider an occurrence r from S_R when it is appended. If both lists are nonempty, the right branch is taken only when

$$r \leq S_L[i].$$

All left values before position i have already been placed and are strictly smaller than r ; all left values from position i onward are at least r . Therefore the profitable cross pairs ending at this occurrence of r are exactly the *placedCount* pairs formed with the already placed left values, and their total profit is

$$\text{placedCount} \cdot r - \text{placedSum}.$$

If S_L is exhausted, every left value has been placed and is strictly smaller than each remaining right value, so the same formula counts all cross pairs ending at every remaining right value. If S_R is exhausted, no right endpoint remains, so appending the remaining left values creates no additional cross profit.

Each profitable pair lies in exactly one of the disjoint classes

$$\text{inside } L, \quad \text{inside } R, \quad \text{crossing from } L \text{ to } R.$$

The algorithm returns

$$\text{profit}L + \text{profit}R + \text{crossProfit} = Q(L) + Q(R) + Q_{\text{cross}} = Q(A).$$

This proves the induction claim.

Running time. The two recursive calls process arrays of size $n/2$. During the merge, each element of S_L and S_R is appended once. Every update of *placedCount*, *placedSum*, and *crossProfit* takes constant time. Thus the nonrecursive work is $\Theta(n)$, and

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Here

$$a = 2, \quad b = 2, \quad f(n) = \Theta(n) = \Theta\left(n^{\log_2 2}\right).$$

This is Case 2 of the Master Theorem, so

$$\boxed{T(n) = \Theta(n \log n)}.$$

For $A = (4, 1, 3, 2, 5)$, the profitable-pair profits are 1, 2, 1, 4, 2, 3, and

$$Q(A) = 1 + 2 + 1 + 4 + 2 + 3 = \boxed{13}.$$